

Technische Hochschule Ingolstadt
Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

Convolutional Neural Networks
CNN for MNIST
Performance Measurement

Oct 16, 2025

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt / Fraunhofer IVI

Autonomous, Cooperating Systems

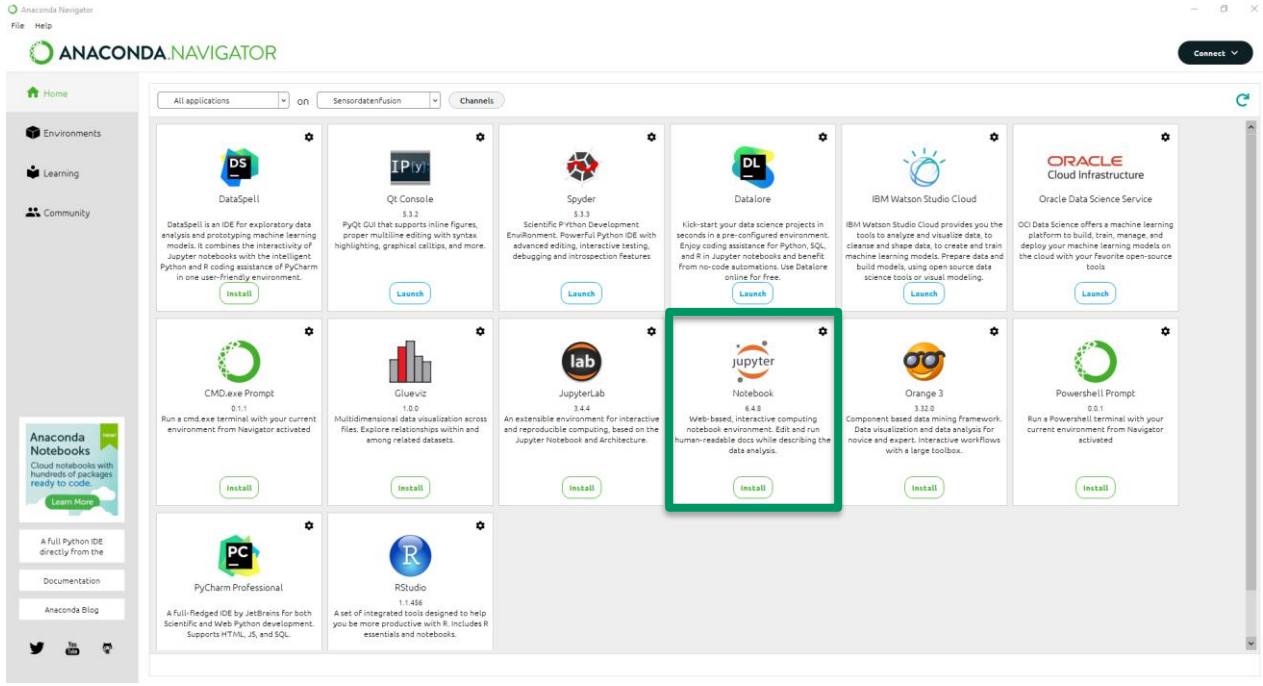
Email: bjoern.giesler@thi.de

- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **News on Moodle:**
  - Slides for the lecture (upcoming lectures marked as WIP and are subject to change!)
  - Formulary, Exercises (Solutions provided after discussion in the lecture)
  - Exercises are optional, one task / sheet per week, but are highly recommended.
    They can be handed in for correction.
  - I will *try* to publish exercises before Wednesday‘s lecture so that initial questions can be discussed on Thursday.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

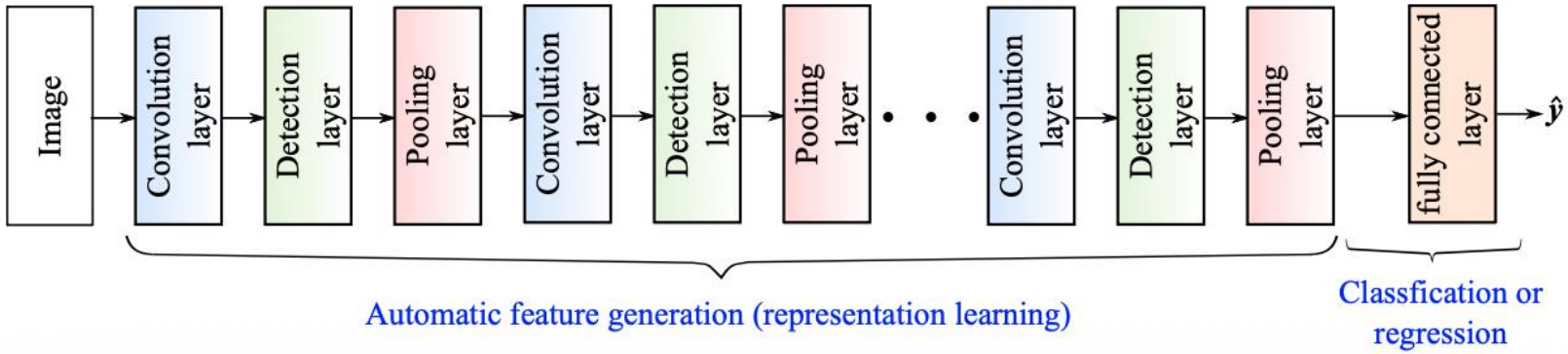
- **You will find two exercises accompanying last week‘s lectures on Moodle.**
- ***Exercise Sheet 1***
  - A couple of very gentle initial exercises (vector/matrix multiplication, Euler angles, derivatives / gradient vectors).
  - Doing this sheet is **highly recommended**, as is handing it in for correction. Depending on number of submissions, I will try to give results on Monday.
- ***ProgExercise 2***
  - A Python program which realizes a Viola Jones face detector.
  - Your task is to use the supplied camera calibration matrix K, and the size of the detected face, to estimate the 3D position of the face in space.
  - This will develop into a longer series of programming exercises estimating positions and velocities of objects in space with Kalman Filters.
  - Doing the exercise therefore is **highly recommended** as well.
- **Solutions for both of the exercises will be posted on Wed Oct 22.**

Software: Python/Anaconda

- For the ML examples we will use today, will use the Jupyter Notebook environment.
- It is recommended you install the Anaconda Python environment and distribution, that installs Jupyter as one of its environments.
- You can find it at https://www.anaconda.com/.



- **Convolutional Neural Networks (ger. Faltungsnetzwerke) are well suited for processing time series or image data, as they are able to find local features in the data and learn them.**
- **CNNs are very similar to multilayer perceptrons, but they require less parameters for the same number of hidden layers.**
- **This is realized by replacing at least one weighting operation $\mathbf{a}^\top \mathbf{x} + b$ by another linear operation: the Convolution or the Deterministic Cross-Correlation, respectively.**



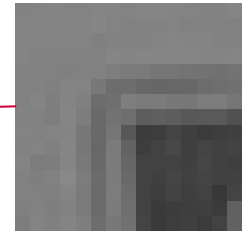
- **Deep CNNs consist of a sequence of convolution, detection and pooling layers, completed by a fully connected layer.**

- **We can think of an image as a 2D function, $\mathrm{Img}(u, v) \rightarrow [0, I_{max}]$, where u, v are the image coordinates and Imax is the maximum intensity allowed by the image.**
- **Object characteristics take the form of local image changes in one or two directions.**
- **We can amplify these changes for later processing by deriving the image in both directions.**
- **The derivative again takes the form of an image.**





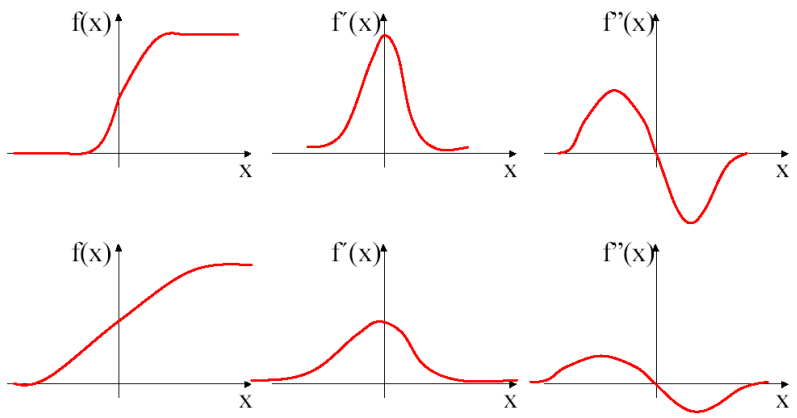
Low change



High change in two directions



High change in one direction



- **The convolution of two discrete D-dimensional signals $v(n_1, n_2, ..., n_D)$ and $w(n_1, n_2, ..., n_D)$ can be written as**

$$x(n_1, n_2, ..., n_D) = v(n_1, n_2, ..., n_D) *\overset{D}{\cdots}* w(n_1, n_2, ..., n_D)$$

$$= \sum_{k_1=-\infty}^{\infty} \sum_{k_2=-\infty}^{\infty} \cdots \sum_{k_D=-\infty}^{\infty} v(n_1 + k_1, n_2 + k_2, \ldots, n_D + k_D) w(k_1, k_2, \ldots, k_D)$$

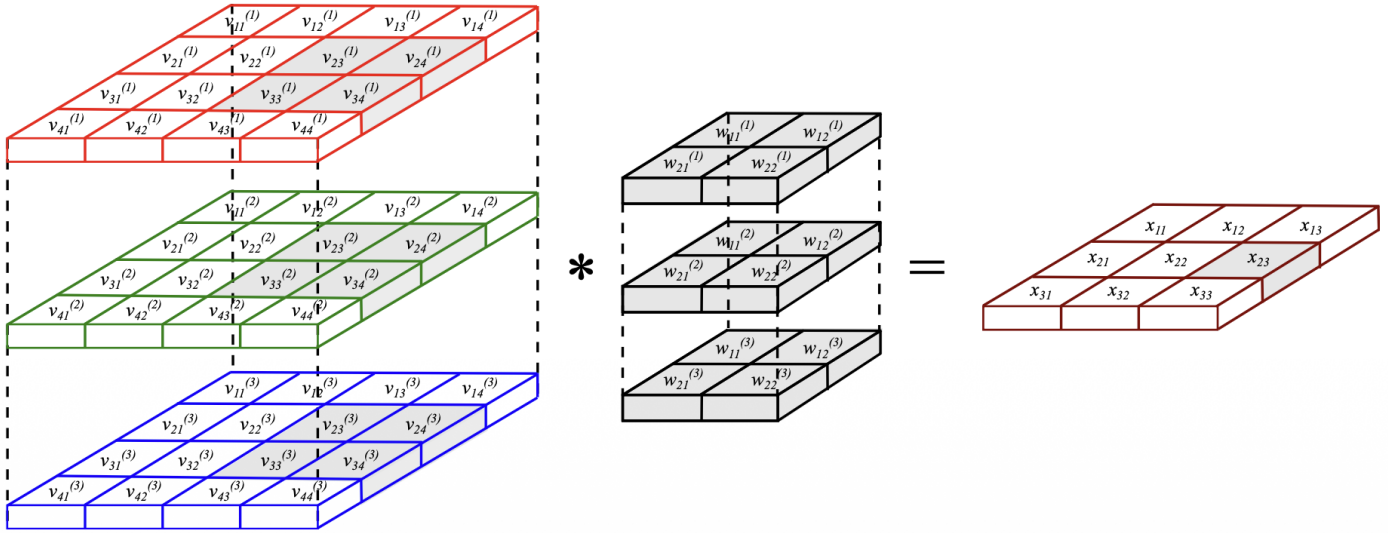
  (technically, that is the cross-correlation, convolution would be $v(n_1 - k_1, n_2 - k_2, \ldots)$, but in computer vision these are typically used interchangeably as they differ only by sign).

- **For D=2, the data structure can be interpreted as an image.**
- **We have seen convolution kernels in lecture 3.**

$$\frac{1}{16}\begin{vmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{vmatrix} \qquad C_{\mathrm{Sx}} = \begin{vmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{vmatrix}, C_{\mathrm{Sy}} = \begin{vmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{vmatrix}$$

- **In CNNs, the kernel is *learned and not predetermined*.**

- **The advantage of the convolution operation is the capturing of local structure in an image, thereby learning the typical required patterns.**
- **In lecture 3 we have discussed that convolution is typically done on a grayscale image after converting RGB to grayscale.**
- **However we can think of RGB images as 3-dimensional data structures with the more general convolution equation.**



Convolution of 4x4x3 image with 2x2x3 kernel and adding up to a 3x3x1 image

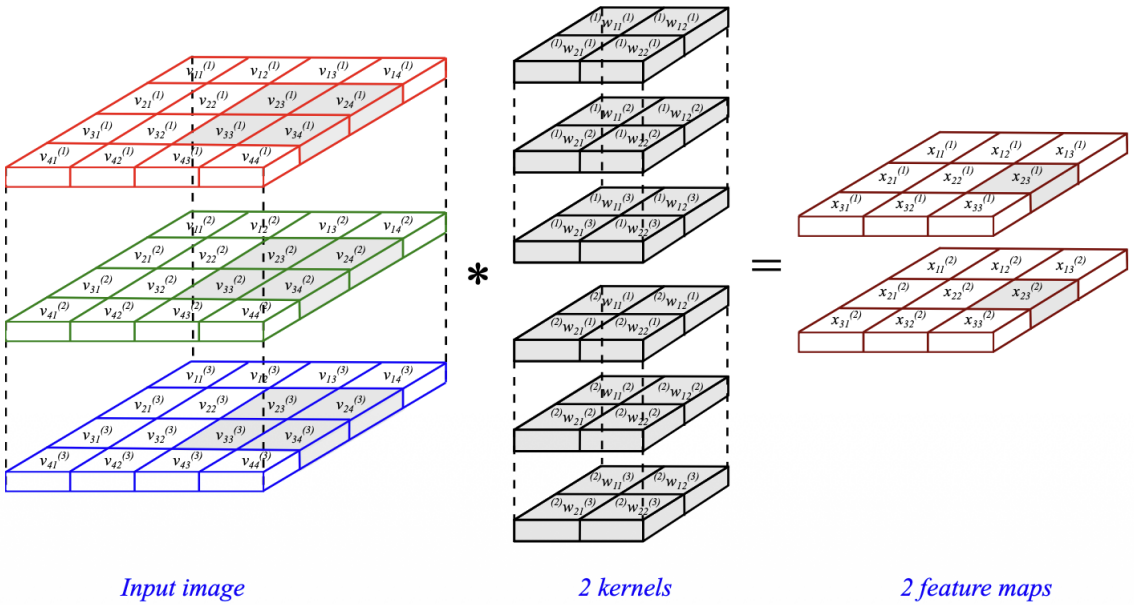
The values $x_{11}, \ldots, x_{33}$ are derived from the convolution of the 4x4x3 image with the 2x2x3 kernel:

$$\begin{aligned} x_{23} = & v_{23}^{(1)} w_{11}^{(1)} + v_{24}^{(1)} w_{12}^{(1)} + v_{33}^{(1)} w_{21}^{(1)} + v_{34}^{(1)} w_{22}^{(1)} \\ & + v_{23}^{(2)} w_{11}^{(2)} + v_{24}^{(2)} w_{12}^{(2)} + v_{33}^{(2)} w_{21}^{(2)} + v_{34}^{(2)} w_{22}^{(2)} \\ & + v_{23}^{(3)} w_{11}^{(3)} + v_{24}^{(3)} w_{12}^{(3)} + v_{33}^{(3)} w_{21}^{(3)} + v_{34}^{(3)} w_{22}^{(3)} \end{aligned}$$

where the high index denotes the R, G, and B channel.

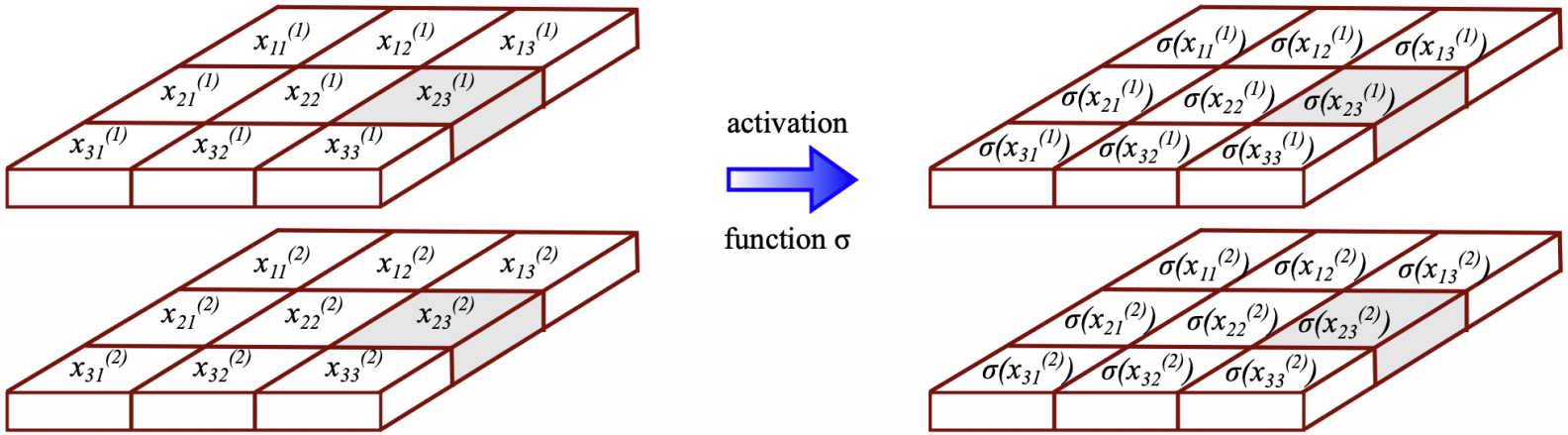
- **We can interpret the w_{ij} as the input weights to a neuron.**
- **In order to dertermine the 9 values $x_{11}, \ldots, x_{33}$ we only need to learn 3*4+1= 13 parameters.**
- **Solving the same task with a perceptron requires 16*3*9+9= 441 parameters.**

- **Usually we do not just generate one feature map from an image, but several.**
- **These are obtained by convolving with different kernels K_F .**



- **So far, we are in the convolution layer of a CNN, where everything is strictly linear.**

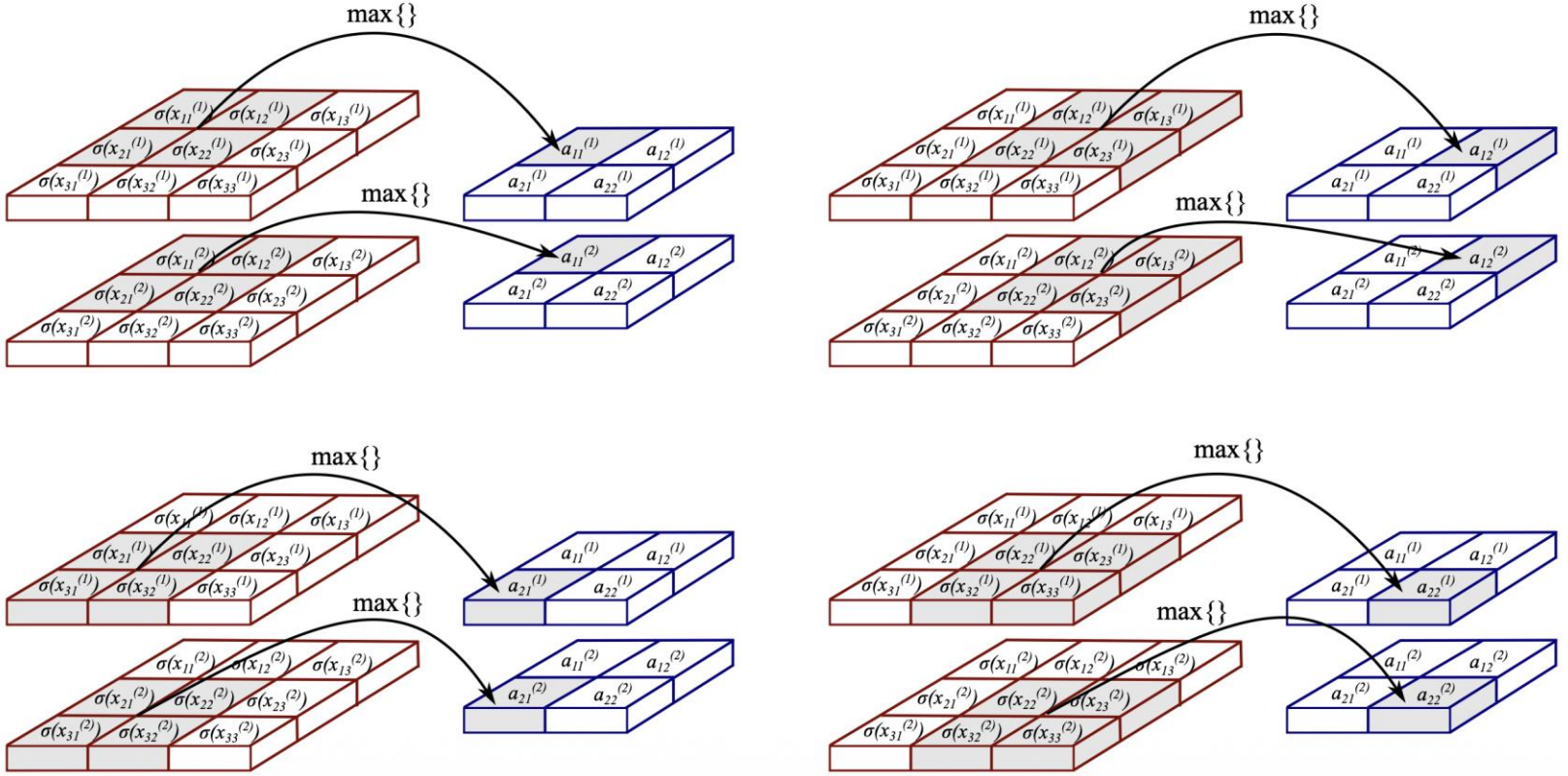
- **Detection Layers introduce nonlinearities to CNNs and are realized with activation functions, as previously discussed in the multilayer perceptron section.**
- **The activation functions affect the feature maps element-wise.**
- **Often the ReLu function, introduced in Subsection 2.1, is used as an activation function.**
- **The dimensions of the input and output keep the same size after the detection layer.**



- **Pooling Layers, often also referred to as downsampling layers, reduce the resolution of the feature map by replacing areas of the feature map with statistical quantities.**
- **Pooling yields an invariance with regard to minor translations of the input, i. e., small local translations in the input data structure lead to no effect at the output of the pooling layer.**
- **Often „Max Pooling“ is used – only the maximum value from a section is kept and all the other values are discarded.**
- **Usually a quadratic region with length and width L_P is chosen, which is slid over the feature map with a defined step size (also named stride) S_P and the covered area is replaced by one statistical quantity, e. g., the maximum value.**

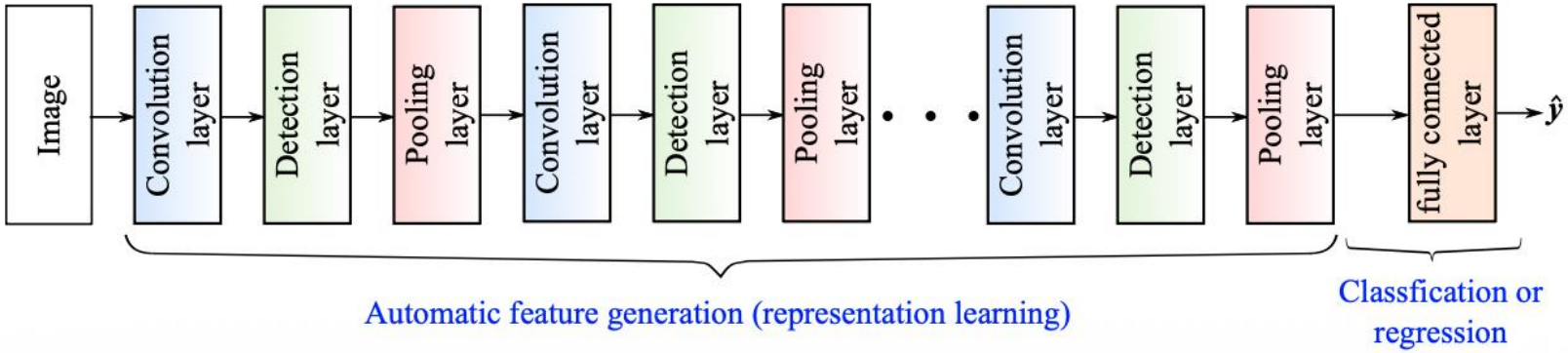
Sensor Networks and Sensor Fusion

Convolutional Neural Networks

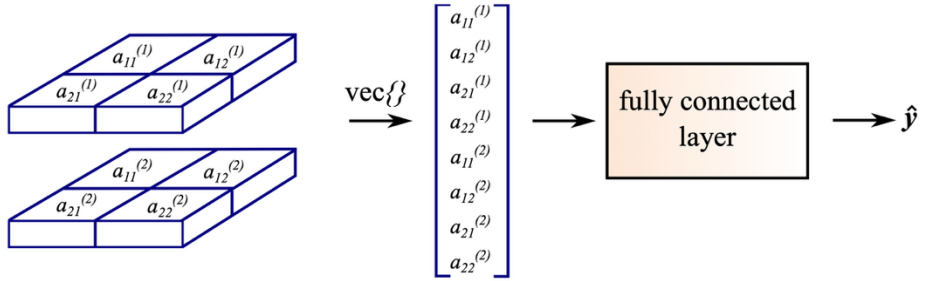


Max Pooling with $L_{\mathrm{P}} = 2, S_{\mathrm{P}} = 1$

- **A CNN usually is set up with several convolution, detection and pooling layers, finally followed by a fully connected layer.**



- **The fully connected layer transforms the final feature maps to the output of the network.**
- **Therefore the feature maps are rearranged to a single vector using the vec operator.**
- **This vector is then the input for the fully connected layer, which computes the output of the CNN.**



- **Backpropagation follows the same general rule for multilayer perceptrons, with the following specialties:**
  - The activation functions in the fully connected layer(s) and the detection layers are differentiated as usual.
  - The convolution layer does not have an activation function; its derivative is simply the image at the given position:

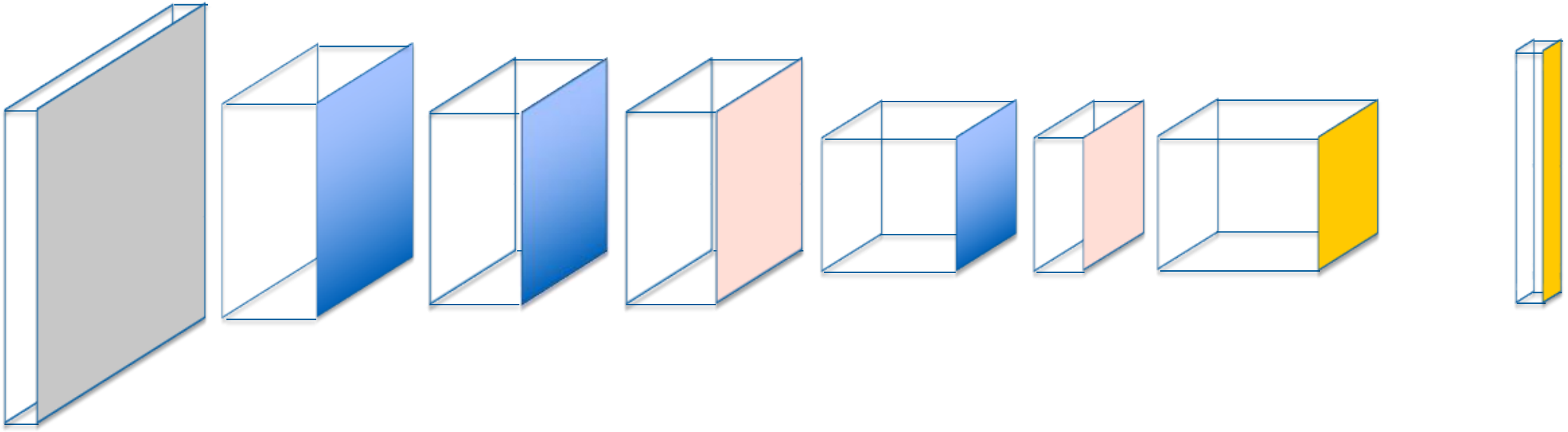
$$\frac{\partial x_{ij}}{\partial w_{kl}} = \frac{\partial}{\partial w_{kl}} \sum_m \sum_n v_{i+m,j+n} w_{mn} = v_{i+k,j+l}$$

  - The statistical function used in the pooling layers may not be differentiable. The maximum function for example is not. We can use a convention (a „subdifferential“):

$$\frac{\partial \max(p,q)}{\partial p} = \begin{cases} 1 & \text{if } p > q \\ 0 & \text{else} \end{cases}$$

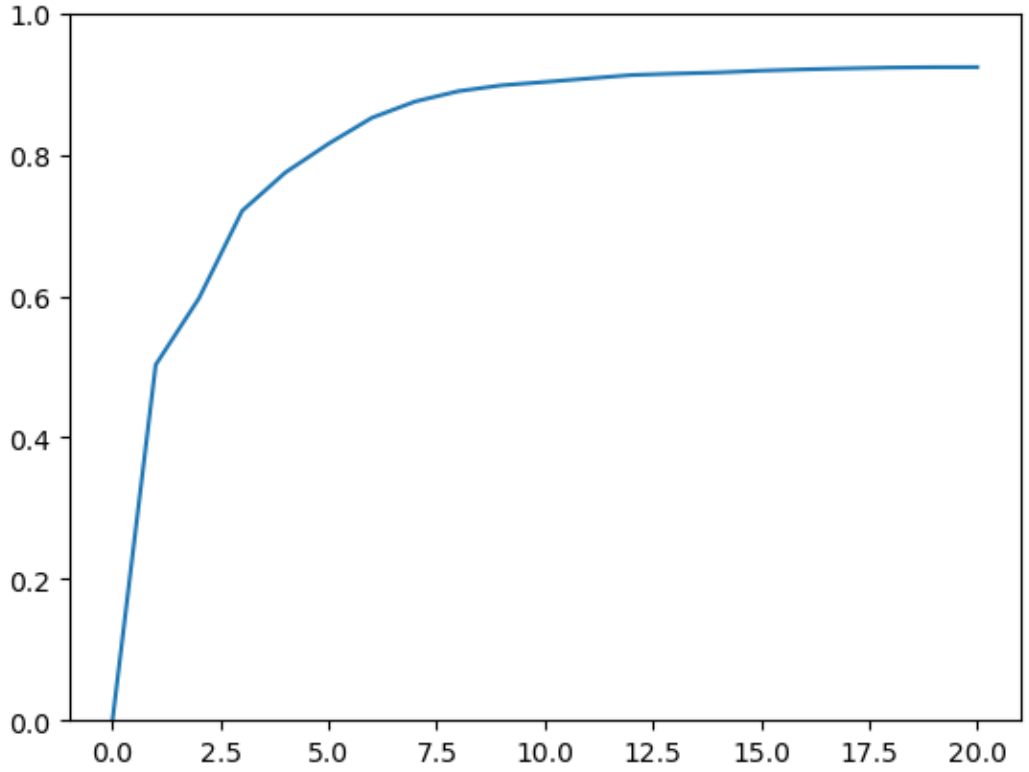
Machine Learning: CNN for MNIST

- **We are revisiting our MNIST training, this time with a CNN.**
- **Network Structure:**

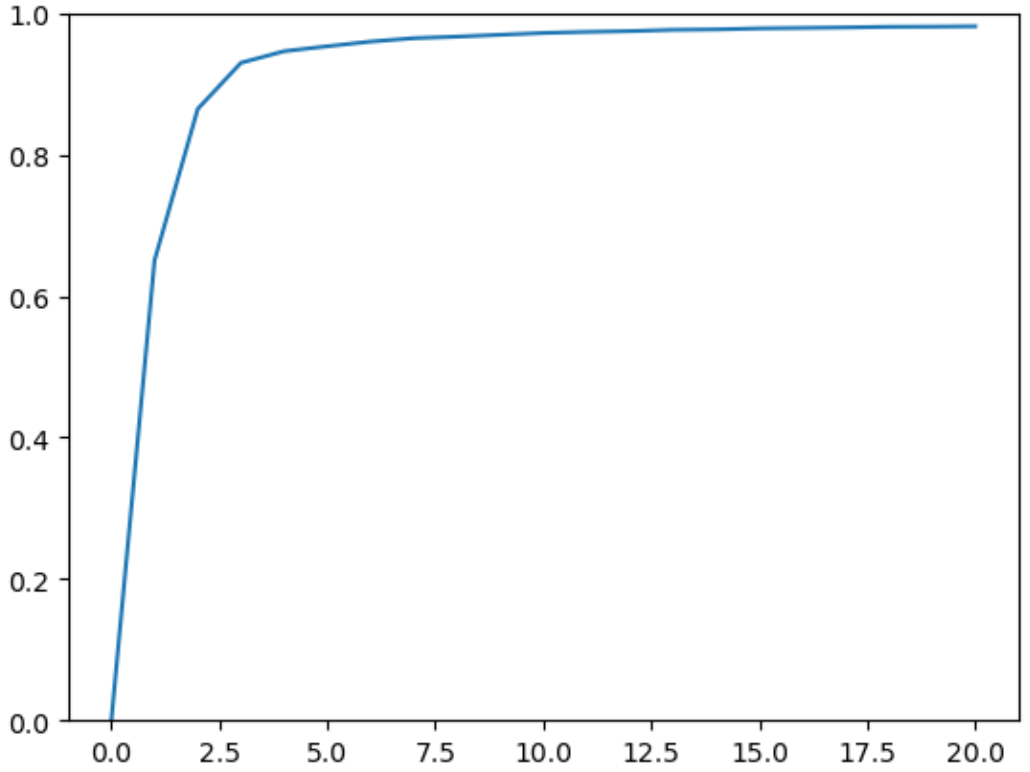


28x28x1 | Conv: 24x24x32 | Conv: 20x20x32 | 2x Maxpool | Conv: 6x6x64 | 2x Maxpool | Fully Connected Layer: 3*3*64 → 256 | Fully Connected Layer: 256 → 10

Comparison of Convergence



ML Perceptron after 20 Epochs: 92.36% accuracy



CNN after 20 Epochs: 98.13% accuracy

- **As we can see, the CNN is much better at recognition**
  - Takes 5 epochs to reach the same quality as the fully connected network does after 20 epochs
  - At the end of training, easily goes up to 99%
- **It is also much slower to train than the FCNN.**
- **There is a great page to visualize CNNs such as this here: https://adamharley.com/nn_vis/**

Machine Learning: Performance Measurements

Measuring Quality

- **Precision and Recall: Classification Quality**
  - ***Precision*** measures *how many classifications are correct*:

$$\text{Precision} = \frac{\#\text{True Positive Classifications}}{\#\text{Total Classifications}} = \frac{tp}{tp + fp}$$

  - ***Recall*** measures *how many instances were identified:*

$$\text{Recall} = \frac{\#\text{True Positive Classifications}}{\#\text{Total Classifiable Objects}} = \frac{tp}{tp + fn}$$

  - Example:
    - An image contains 10 cars.
    - 7 were correctly identified.
    - 2 non-car patterns were classified as cars (False Positives).
    - 3 cars were not identified (False Negatives).

Instance Is Detected \ Instance Exists	Yes	No
Yes	True Positive (TP)	False Positive (FP)
No	False Negative (FN)	True Negative (TN)

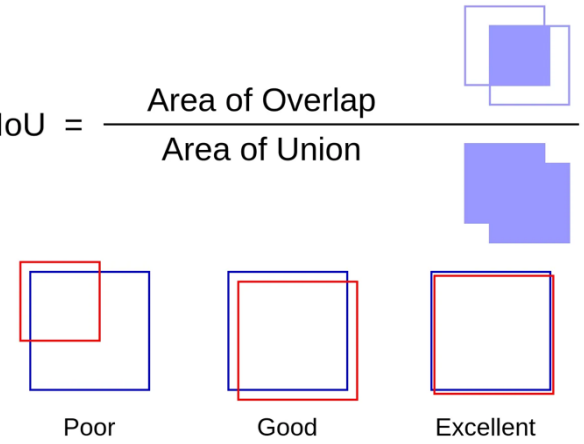
$$\text{Precision} : (7)/(7+2) = 77.8\%$$
$$\text{Recall} : 7/10 = 70\%$$

Measuring Quality

- ***F1 Score*: Classification Quality**
  - The F1 score is the *harmonic mean* of Precision and Recall: $F = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
  - It unifies Precision and Recall into one „goodness score“.

- **Intersection Over Union: Spatial Localization Quality**
  - Measures the normalized overlap of two bounding boxes (e.g. ground truth and prediction).
  - Gives a good estimate on precision of the bounding box extraction, vital for object **spatial localization** .

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



The Trouble with CNNs…

- **CNNs are great to classify objects based on appearance, and they are a lot cheaper than regular Multilayer Perceptrons.**
- **But they are still locked to a certain region given by the number of connections / weights to the fully connected layer.**
- **Pooling layers introduce a bit of scale invariance, but you still need regions of interest.**



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!